

C#基础语法（2）课堂练习

课堂练习

厦门大学信息学院 赵江声 2025





一、数值和转换

1 C#的数值和内置类型

2 数值类型转换

1.1 C#的数值和内置类型



学习如何使用C#的数值类型，以及如何枚举C#的常用数据类型。

- 目标1：给定一个类型，输出他的.NET类型
- 目标2：枚举一个由值类型装箱的Object类型对象，确认他的基本类型



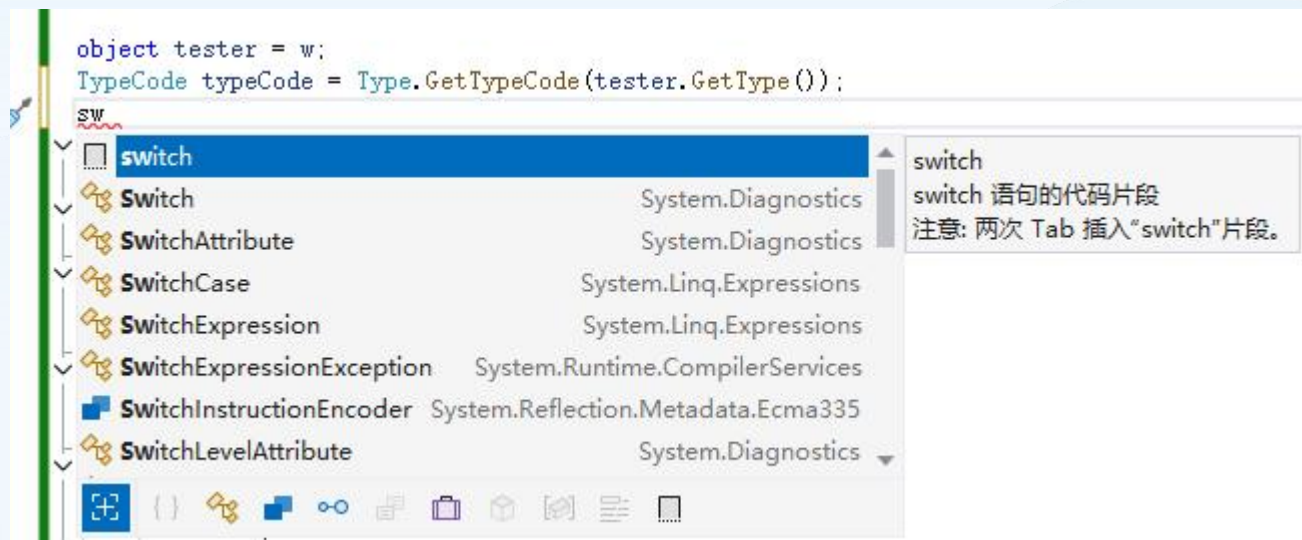
1.1 C#的数值和内置类型

由于switch语句只能枚举常量。所以，C#中提供一个名为TypeCode的枚举值供程序员枚举基本数据类型。

```
// 目标2: 枚举一个由值类型装箱的Object类型对象，确认他的基本类型
object w = 1, x = 1M, y = 1D, z = 1L;

object tester = w;
TypeCode typeCode = Type.GetTypeCode(tester.GetType());
switch (typeCode)
{
    case TypeCode.Empty:
        break;
    case TypeCode.Object:
        Console.WriteLine("Object: " + tester);
        break;
    case TypeCode.DBNull:
        Console.WriteLine("DBNull: " + tester);
        break;
    case TypeCode.Boolean:
        Console.WriteLine("Boolean: " + tester);
        break;
    case TypeCode.Char:
        Console.WriteLine("Char: " + tester);
        break;
    case TypeCode.SByte:
        Console.WriteLine("SByte: " + tester);
        break;
    case TypeCode.Byte:
        Console.WriteLine("Byte: " + tester);
        break;
    case TypeCode.Int16:
        Console.WriteLine("Int16: " + tester);
        break;
}
```

在输入这段代码时，一定要用VS的代码段功能。在输入switch时，看见下图提示时，按下TAB。



1.1 C#的数值和内置类型

之后，在黄色区域（下图是选中状态，取消选中就可以看见黄底）里，将默认变量名改为一个枚举对象。

```
switch (switch_on)
{
    default:
}
```

switch代码段生成时

在输入枚举对象后，系统会自动为你生成switch里所有选项的case语句。不会有任何遗漏。

```
switch (typeCode)
{
    case TypeCode.Empty:
        break;
    case TypeCode.Object:
        break;
    case TypeCode.DBNull:
        break;
    case TypeCode.Boolean:
        break;
    case TypeCode.Char:
        break;
    case TypeCode.SByte:
        break;
    case TypeCode.Byte:
        break;
    case TypeCode.Int16:
        break;
    case TypeCode.UInt16:
        break;
    case TypeCode.Int32:
        break;
    case TypeCode.UInt32:
        break;
    case TypeCode.Int64:
        break;
    case TypeCode.UInt64:
        break;
    case TypeCode.Single:
        break;
    case TypeCode.Double:
        break;
    case TypeCode.Decimal:
        break;
    case TypeCode.DateTime:
        break;
    case TypeCode.String:
        break;
    default:
        break;
}
```

1.2 类型转换

实现以下功能：

- 目标1：用户任意输入一个字符串（必定是TypeCode枚举里的一种）
- 目标2：用户任意输入一个类型名（.NET类型名，关键字均可）
- 目标3：将用户输入的字符串，转换为对应类型的对象

注：这里的类型必须支持TypeCode枚举里的所有类型。DateTime类型表达为2024-9-1 10:20:20.222。即“年-月-日 时:分:秒.毫秒”。





二、结构体和枚举

1 结构体

2 枚举

2.1 结构体

学习使用C#的结构体。

- 目标1：使用结构体，构造一个二维向量
- 目标2：实现向量的相加，相减，标量乘法，向量积和相除
- 目标3：为相加，相减，标量乘法和相除提供运算符重载



2.1 结构体

在类和结构体中可以提供运算符重载，以更加简便的形式书写运算。

```
Vector2D v2d = new()
{
    X = 1,
    Y = 2
};

Console.WriteLine(v2d * 3);
Console.WriteLine(3 * v2d);

5 个引用
public struct Vector2D
{
    public double X;
    public double Y;

    1 个引用
    public static Vector2D operator * (Vector2D v1, double f)
    {
        return new Vector2D()
        {
            X = v1.X * f,
            Y = v1.Y * f,
        };
    }

    0 个引用
    public override string ToString()
    {
        return "(" + X + ", " + Y + ")";
    }
}
```

运算符重载是有顺序的，第一个参数必是左值，第二个参数必是右值。

在左图的例中，`3 * v2d`的左值是`double`，右值是`Vector2D`。与重载的顺序正好相反，所以出错了。对于满足交换律的乘法运算，在重载时必须有两个函数分别对应着两种左值的情况。

即，必须向下图这样书写：

```
1 个引用
public static Vector2D operator * (Vector2D v1, double f)
1 个引用
public static Vector2D operator *(double f, Vector2D v1)
```

思考：向量加法一样满足交换律。但需要写两个重载函数吗？

2.2 枚举

通过以下目标，学习枚举的高阶应用。

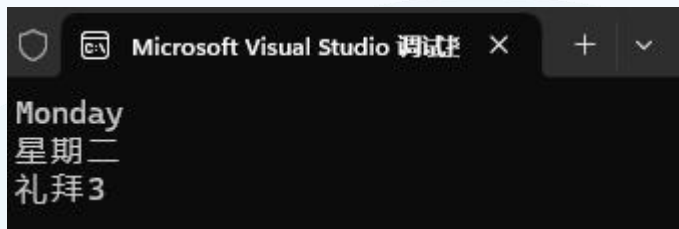
- 目标1：学会枚举的ToString的值
- 目标2：学会使用枚举对应的数字值
- 目标3：学会使用Flags形式的枚举值
- 目标4：将对应的数字值转换为枚举值
- 目标5：使用foreach循环枚举的值或者项



2.2 枚举

目标1：编写以下代码，查看输出值

```
Console.WriteLine (DayOfWeek.Monday);  
Console.WriteLine (DayOfWeek.星期二);  
Console.WriteLine (DayOfWeek.礼拜3);  
  
3 个引用  
public enum DayOfWeek : int  
{  
    Monday = 1,  
    星期二 = 2,  
    礼拜3 = 3,  
}
```



Microsoft Visual Studio 调试 × + ▾
Monday
星期二
礼拜3

枚举直接输出为字符串时，输出的内容就是枚举项的名称。编写仅在一种语言使用的应用程序时，活用中文枚举名可以极大减少工作量。

比如，DayOfWeek枚举，如果该程序仅在中国使用，那么枚举项直接用中文，在输出时就会非常方便。否则必须使用switch来枚举每一项的中文名。

2.2 枚举

目标2：编写以下代码，查看输出值

```
Console.WriteLine((int)DayOfWeek.Monday);  
Console.WriteLine((int)DayOfWeek.星期二);  
Console.WriteLine((int)DayOfWeek.礼拜3);  
  
3 个引用  
public enum DayOfWeek : int  
{  
    Monday = 1,  
    星期二 = 2,  
    礼拜3 = 3,  
}
```

在一些时候，枚举值需要进行数值运算。这时只需要将对象强制转换为基底类型即可。

枚举的基底类型默认为int，并且仅能是byte, sbyte, short, ushort, int, unit, long或者ulong的其中一种。

思考：

1. 如果不写每个枚举项对应的值时，系统默认给的值是什么？
2. 如果第一个值是5，其他值不写，会发生什么？
3. 如果第一个值是70，第二个值是2，第三个值是什么？
4. 如果给的值顺序错乱会发生什么？

2.2 枚举

目标3：编写以下代码，查看输出值

```
Console.WriteLine((int)DayOfWeek.Monday);  
Console.WriteLine((int)DayOfWeek.星期二);  
Console.WriteLine((int)DayOfWeek.礼拜3);  
Console.WriteLine((int)DayOfWeek.星期四);  
Console.WriteLine((int)DayOfWeek.星期五);
```

```
[Flags]  
5 个引用  
public enum DayOfWeek  
{  
    Monday,  
    星期二,  
    礼拜3,  
    星期四,  
    星期五  
}
```

0
1
2
3
4

在枚举上，加Flags标签，并且所有项都不写默认值。则系统按0，1，2，4，8，16的顺序给值。

这种枚举称为：标志枚举

2.2 枚举

目标3：编写以下代码，查看输出值

```
DayOfWeek d = DayOfWeek.Monday | DayOfWeek.星期二;  
Console.WriteLine((int)d);  
Console.WriteLine((int)(d & DayOfWeek.星期二));  
Console.WriteLine((d & DayOfWeek.星期二) == DayOfWeek.星期二);  
Console.WriteLine(d.HasFlag(DayOfWeek.星期二));
```

7 个引用

```
public enum DayOfWeek  
{  
    Monday = 2,  
    星期二 = 1,  
    礼拜三,  
    星期四,  
    星期五  
}
```

Microsoft Visual Studio 调试

```
3  
1  
True  
True
```

标志枚举可以用“按位或”运算组合多个枚举项；

标志枚举可以用“按位与”运算测试某一项是否在组合枚举中出现；

HasFlag是“按位与”运算的简化版本。

思考：

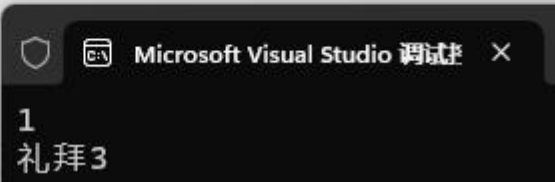
1. 为什么可以用“按位或”组合多个枚举？
2. 为什么可以用“按位与”测试枚举是否出现？
3. 如果标志枚举如上图一样给了错乱的枚举值时会发生什么？
4. 如果标记枚举隔一项给一个错乱的值会发生什么？

2.2 枚举

目标4：编写以下代码，查看输出值

```
Console.WriteLine((int)Enum.Parse(typeof(DayOfWeek), "星期二"));
Console.WriteLine(Enum.Parse(typeof(DayOfWeek), "2"));

2 个引用
public enum DayOfWeek
{
    Monday,
    星期二,
    礼拜3,
    星期四,
    星期五
}
```



枚举的转换为基本数据类型完全不同；
枚举可以用项的名称转换；
枚举可以用基底值对应的字符串转换（但不能直接用基底值，必须转换为字符串才行）。

2.2 枚举

目标4：编写以下代码，查看输出值

```
foreach (var item in Enum.GetValues(typeof(DayOfWeek)))  
{  
    Console.WriteLine(item + ": " + (int)item);  
}  
  
1 个引用  
public enum DayOfWeek  
{  
    Monday,  
    星期二,  
    礼拜3,  
    星期四,  
    星期五  
}
```



思考：var对应的是哪一种数据类型？

2.2 枚举

目标4：编写以下代码，查看输出值

```
foreach (var item in Enum.GetNames(typeof(DayOfWeek)))  
{  
    Console.WriteLine(item);  
}  
  
1 个引用  
public enum DayOfWeek  
{  
    Monday,  
    星期二,  
    礼拜3,  
    星期四,  
    星期五  
}
```



思考：var对应的是哪一种数据类型？

感谢观看

单击此处添加文档副标题内容

